

Chapter 5 – OAuth 2.0

Learning Objectives

After completing this chapter, you should be able to:

- Understand why OAuth exists.
 - Explain the OAuth 2.0 authorization flow.
 - Differentiate OAuth from SAML and OIDC.
 - Understand Access Tokens and Refresh Tokens.
 - Explain OAuth Scopes.
 - Understand Consent Screens.
 - Explain Domain-Wide Delegation.
 - Troubleshoot common OAuth issues.
 - Apply the Principle of Least Privilege.
-

1. Business Problem

Imagine a company uses an AI meeting assistant.

The application needs to:

- Create meeting notes
- Save notes to Google Drive

Without OAuth, the user would have to give the application their Google password.

Problems:

- The application knows the user's password.
- Password compromise affects the entire account.
- Password changes break the integration.
- The application gets unrestricted access.

This is insecure.

OAuth solves this problem.

2. What is OAuth?

OAuth 2.0 is an authorization framework that allows an application to access specific resources on behalf of a user without knowing the user's password.

The user authenticates with Google.

Google issues an **Access Token** to the application.

The application uses the token—not the password—to call Google APIs.

Real Enterprise Example

Company

Contoso Ltd.

Application

Expense Management System

Requirements

- Read employee profile
- Create Drive folder
- Upload receipts

The application requests only the permissions it needs.

Employees never share their Google password with the application.

3. OAuth Roles

OAuth defines several participants.

Component	Purpose
Resource Owner	The user who owns the data
Client	The application requesting access
Authorization Server	Google Identity service that issues tokens

Component	Purpose
Resource Server	Google API that stores the protected resource

Example

Employee

↓

Expense Application

↓

Google Authorization Server

↓

Google Drive API

The application receives permission only after user consent.

4. OAuth Authorization Flow

User

↓

Application requests Drive access

↓

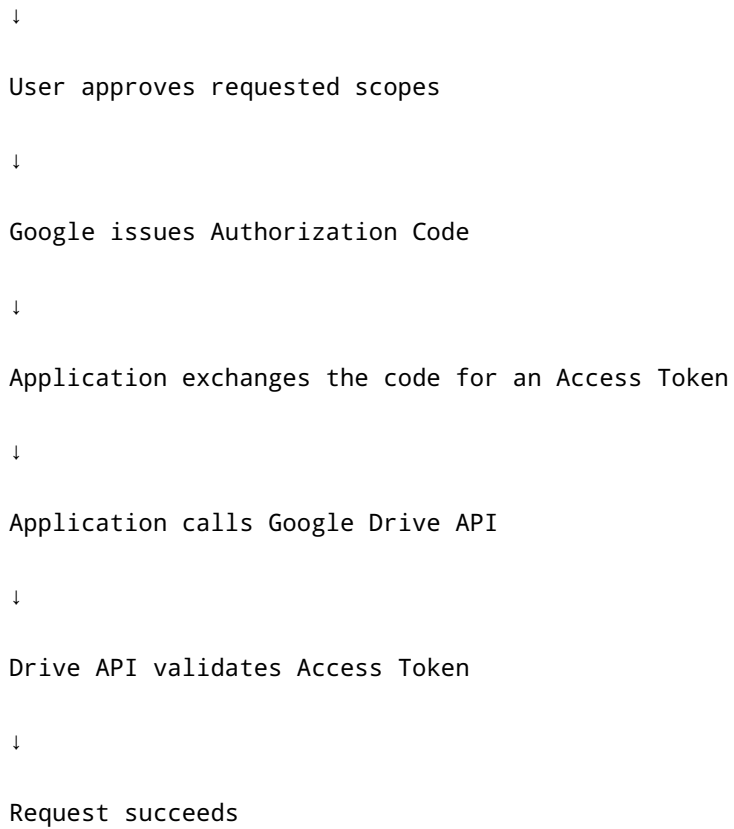
Redirect to Google

↓

User authenticates

↓

Google shows Consent Screen



Notice:

The application never receives the user's password.

5. Consent Screen

Before an application receives access,

Google informs the user:

"This application is requesting the following permissions."

Example:

- View your profile
- Create Drive files
- Read Calendar events

The user decides whether to allow access.

Example

Application

Meeting Notes AI

Requested Scopes

✓ Create Drive files

✓ Read Calendar events

The application does **not** request:

✗ Read Gmail

✗ Read Contacts

This follows the Principle of Least Privilege.

6. OAuth Scopes

A scope defines what an application is allowed to do.

Think of a scope as a permission.

Examples

Scope	Permission
Drive File	Create and manage files created by the app
Drive Readonly	Read Drive files
Gmail Readonly	Read Gmail messages
Gmail Send	Send emails
Calendar	Manage calendar events
Admin SDK Directory	Manage users and groups

Applications should request only the scopes they require.

Example

AI Note-Taking Application

Purpose

Create meeting notes.

Required Scope

✓ Drive File

Not Required

✗ Gmail Read

✗ Calendar Read

✗ Offline Access

Requesting unnecessary scopes increases security risk.

7. Access Token

An Access Token is a temporary credential that authorizes API access.

Think of it as a temporary visitor badge.

Characteristics

- Short-lived
- Sent with API requests
- Represents granted permissions
- Can expire

Example

Application

↓

Access Token

↓

Google Drive API

↓

Access Granted

8. Refresh Token

Access Tokens expire.

A Refresh Token allows the application to obtain a new Access Token without requiring the user to sign in again.

Flow

Access Token expires

↓

Application sends Refresh Token

↓

Google issues new Access Token

↓

Application continues working

Refresh Tokens are typically long-lived and must be protected carefully.

Access Token vs Refresh Token

Access Token	Refresh Token
Used to access APIs	Used to obtain a new Access Token
Short-lived	Long-lived
Sent with API requests	Not sent to Resource APIs

Access Token	Refresh Token
Expires frequently	Used only with the Authorization Server

9. OAuth Scopes in Enterprise Design

Suppose an HR application needs to:

- Create users
- Suspend users
- Update user information

Required Scope

✓ Admin SDK Directory API

Not Required

✗ Gmail

✗ Calendar

✗ Drive

Least privilege reduces risk.

10. Domain-Wide Delegation (DWD)

Normally,

a user must grant consent.

Enterprise automation is different.

Example

HR onboarding automation

Every day:

- Create users
- Suspend users

- Add users to groups

Waiting for each employee to approve consent is impractical.

Instead,

an administrator grants **Domain-Wide Delegation** to a service account.

The service account can then act on behalf of users within the approved scopes.

Example

Service Account

↓

Domain-Wide Delegation

↓

Admin SDK Directory API

↓

Create User

↓

Add User to Group

↓

Suspend User

No interactive user consent is required for each operation.

11. Least Privilege

Suppose an application only needs to create files in Drive.

Correct

✓ Drive File

Incorrect

✗ Full Drive Access

✗ Gmail Read

✗ Calendar Read

✗ Contacts

Always grant the minimum permissions necessary.

12. Multiple Service Accounts

Enterprise best practice:

One service account

↓

One business function

Example

Service Account A

Purpose

User provisioning

Scope

Admin SDK Directory API

Service Account B

Purpose

Send welcome emails

Scopes

Gmail API

Service Account C

Purpose

Generate reports

Scopes

Reports API

Drive API

If one service account is compromised,
the attacker gains access only to that business function.

13. OAuth Troubleshooting

Application cannot access Drive.

Questions to ask:

1. Is the user authenticated?
 2. Was consent granted?
 3. Is the Access Token valid?
 4. Has the token expired?
 5. Is the required scope present?
 6. Is the API enabled?
 7. Is Domain-Wide Delegation configured correctly (if applicable)?
-

Common OAuth Errors

invalid_scope

Cause

Application requested a scope that is invalid or not allowed.

Resolution

Review the requested scopes.

invalid_token

Cause

Access Token expired or is malformed.

Resolution

Obtain a new Access Token using the Refresh Token.

insufficient_permissions

Cause

Token exists,

but it lacks the required scope.

Resolution

Request the correct scope and obtain a new token.

consent_required

Cause

User has not granted consent.

Resolution

Complete the OAuth consent flow.

403 Forbidden

Cause

User is authenticated,
but not authorized to perform the requested action.

Example

A service account has Domain-Wide Delegation,
but the Admin SDK scope has not been authorized in the Admin Console.

Result

Google returns:

403 Forbidden

OAuth vs SAML

OAuth	SAML
Authorization framework	Authentication protocol
Access to APIs	Single Sign-On
Uses Access Tokens	Uses SAML Assertions
Designed for delegated access	Designed for enterprise authentication

Remember:

SAML proves identity.

OAuth grants permissions.

Best Practices

- ✓ Request only the required scopes.
 - ✓ Review third-party applications regularly.
 - ✓ Audit Domain-Wide Delegation.
 - ✓ Separate service accounts by business function.
 - ✓ Rotate service account keys if they are used (or prefer keyless approaches where possible).
 - ✓ Remove unused OAuth applications.
-

Common Mistakes

- ✗ Treating OAuth as an authentication protocol.
 - ✗ Requesting excessive scopes.
 - ✗ Using one service account for every automation.
 - ✗ Forgetting to review Domain-Wide Delegation.
 - ✗ Ignoring expired Access Tokens.
-

Interview Questions

1. What problem does OAuth solve?
2. What is an Access Token?
3. What is a Refresh Token?
4. What is an OAuth Scope?
5. Why is user consent required?
6. What is Domain-Wide Delegation?
7. Why should service accounts follow the Principle of Least Privilege?

8. How would you troubleshoot a 403 Forbidden error from the Admin SDK?

Key Takeaways

OAuth does **not** authenticate users by itself.

OAuth authorizes applications.

Access Tokens grant temporary access.

Refresh Tokens obtain new Access Tokens.

Scopes define permissions.

Domain-Wide Delegation enables enterprise automation.

Least privilege should guide every OAuth design.

Chapter Summary

In this chapter you learned:

- ✓ Why OAuth exists.
- ✓ OAuth roles and authorization flow.
- ✓ Consent Screens.
- ✓ OAuth Scopes.
- ✓ Access Tokens.
- ✓ Refresh Tokens.
- ✓ Domain-Wide Delegation.
- ✓ Least Privilege.
- ✓ Common OAuth errors and troubleshooting.

The next chapter covers **OpenID Connect (OIDC)**, which builds on OAuth 2.0 to provide authentication using ID Tokens, allowing applications to verify a user's identity while still leveraging OAuth's authorization capabilities.